

Lab exercises

Track 16.5 — 📁 LangSmith · 16-ai-tech-stack/tracks/langsmith/lab/EXERCISES.md

Lab — LangSmith: Tracing, Evals & Observability

Work top to bottom. Every exercise states a **deliverable**; if you cannot produce it, you have not finished. Everything through §6 runs **offline with no API key**. §7 is optional and needs a free account.

1. Warm-up: read the tracer you were given (45 min)

Run `../code/minimal_tracer.py` and answer from the output:

- **1a.** `support_agent` shows ~120ms of latency but \$0.0019 of cost, while `chat_completion` shows ~93ms and the *same* \$0.0019. Explain, in one sentence, why the parent's cost is not additional.
- **1b.** The second trace contains a `lookup_order` run with `ERR TimeoutError`, yet the trace is not marked failed and the user got an answer. Name one real incident shape this hides, and one filter you would add to catch it.
- **1c.** Set `SAMPLING_RATE = 0.5` and run five times. Count roots per run. Then explain what would break if the sampling decision were made *per-span* instead of at the root.

Deliverable: three sentences, one per part.

2. Break the contextvar (1h)

The decorator nests runs by tracking the current run in a `contextvars.ContextVar`. Find where that breaks.

- **2a.** Call `support_agent` from inside a `concurrent.futures.ThreadPoolExecutor`. Does nesting survive?
- **2b.** Do the same with `multiprocessing.Pool`. What happens to the tree, and why?
- **2c.** Make a `@traceable` function a **generator** that yields three values. When does `end_ns` get set relative to the caller consuming the generator? What does that do to the reported latency?
- **2d.** Fix exactly one of the three, using the pattern the real SDK uses for the same problem (a manually constructed run tree with an explicit parent id).

Checks: for 2a nesting should survive (contextvars copy into threads); for 2b it should not. You should be able to state the rule in one line: *the contextvar follows the context, not the call*.

Deliverable: a table of the three cases — `nests` / `orphans` / `wrong timing` — and your fix.

3. Cost archaeology (45 min)

- **3a.** In `minimal_tracer.py`, change `call_llm`'s default `tier` from `"mid"` to `"frontier"` and re-run. Report the change in projected cost at 1M traces.
- **3b.** Now leave the tier alone and instead make `retrieve` return `k=8` documents that each get concatenated into the prompt (roughly triple the prompt length). Report the cost change.
- **3c.** Which change would your current monitoring have caught, and which would have been invisible? Design the one alert that catches **both**, and state its threshold.

Deliverable: three numbers, and one alert definition in the form `alert when <metric> <comparator> <threshold> over <window>`.

4. Write an evaluator that earns its place (1.5h)

Open `../code/eval_harness.py`.

- **4a.** `exact_match` scores 0.00 on every row of both experiments. Write two sentences on why it was still worth including in the harness.
- **4b.** Add a `cites_policy` evaluator: the answer must contain a policy number matching `POL-\d{4}` whenever the reference output mentions a policy. Make it return `None` (not `0.0`) when the example has no policy — an evaluator that does not apply must abstain, not fail. Confirm the aggregation code does not crash on `None`, and fix it if it does.
- **4c.** Add `refusal_rate` as a **summary** evaluator (whole-experiment, not per-row).
- **4d.** Delete the preamble from `app_v2` and re-run. Report the change in `naive_length_judge` and in `rubric_judge`. Which number moved more, and what does that tell you about what the naive judge was measuring?

Checks: after 4d, `naive_length_judge` should collapse toward `v1`'s score while `fact_coverage` stays put. That gap is verbosity bias, measured.

5. Make the gate do its job (1h)

- **5a.** Run `python eval_harness.py --update-baseline`, then `--strict`. Confirm exit code 1 and explain which check you would actually block a merge on.
- **5b.** Introduce a *silent* regression: make `app_v1` drop the word "original" from the refund fact. Does the gate catch it? Which evaluator fires? If none do, add one.
- **5c.** Tighten `MAX_DROP` to `0.001` and re-run three times. Describe the failure mode you just created. (Hint: `num_repetitions` exists for a reason.)
- **5d.** Write the GitHub Actions step you would actually use, including which dataset split runs on every commit versus on every PR, and a one-sentence justification of the split boundary.

Deliverable: a working `.github/workflows/evals.yml` snippet and your answer to 5c.

6. Design review: instrument a real system (2h) — the capstone

Pick an LLM app: yours, or build a three-step one (retrieve → generate → validate).

Produce a single document containing:

1. **A trace design.** Every span you will emit, its `run_type`, and what goes in `inputs/outputs`. Include the shape of one *failing* trace.
2. **A metadata contract.** Which keys go on every run (release SHA, `thread_id`, tenant, feature flag) and who owns each.
3. **A sampling and retention policy.** Rate for healthy traffic, rate for errors, retention window — and the cost estimate that justifies the numbers.
4. **A PII decision record.** What is redacted, at which of the four seams (`process_inputs`, client `anonymizer`, client `hide_inputs`, `LANGSMITH_HIDE_*`), what debugging value you knowingly gave up, and who signed off. Cross-reference `../../../../13-ai-security/` and `../../../../12-ai-governance/`.
5. **A dataset plan.** The three trace filters you will harvest from, the splits, and how a failure becomes an example without anyone remembering to do it.
6. **An eval suite.** Three evaluators minimum, with one sentence each on what would make it *wrong*.
7. **A judge calibration plan.** Sample size, who labels, what agreement number you accept, when you re-check.
8. **Five alerts**, each with a metric, threshold, window, and the name of the person it pages.
9. **A build-vs-buy paragraph** naming the tool you would choose from §9 of `../notes/02-evaluation-and-operations.md`, and what you are giving up by choosing it.

Check: hand it to a colleague. If they can implement §1–3 without asking you a question, and they can name one thing your design will *not* catch, you passed.

7. Optional: the real thing (1h, needs a free account)

- **7a.** Sign up, set `LANGSMITH_TRACING=true` and `LANGSMITH_API_KEY`, and run `python ../code/langsmith_instrumentation_reference.py --live`. Find your trace.
 - **7b.** Deliberately reproduce the classic bug: unset `LANGSMITH_TRACING` but keep the API key. Confirm that nothing errors and nothing appears. Write down how long it would have taken you to find this.
 - **7c.** Instrument a raw `openai` or `anthropic` script with `wrap_*` and `@traceable`, using **no LangChain at all**. Confirm the trace renders correctly. This is the exercise that kills the "LangSmith only works with LangChain" belief.
 - **7d.** Create a dataset from three of your own traces, run `evaluate()` with one evaluator, and open the experiment's traces. Then add one run to an annotation queue and score it against a rubric you wrote.
-

8. Stretch: swap the backend (2h)

Take your instrumented app from §6 and emit **OpenTelemetry** spans using the GenAI semantic conventions instead of the LangSmith SDK. Send them to LangSmith via OTLP, then to a self-hosted Langfuse or Phoenix.

Check: the same application code, unchanged, produces usable traces in two backends. Write one paragraph on what you lost in the translation — and be specific, because "nothing" is the wrong answer.